

Datavalidering för tabulär data med Pandera

Syfte

Målet med den här fördjupningen har varit att undersöka hur biblioteket Pandera kan användas för att bygga ett automatiserat valideringsflöde för tabulär data i Python (Pandera Documentation, u.å.).

I utbildningen har vi pratat mycket om datastädning – att se till att datatyper stämmer och att det inte finns saknade värden. Mitt projekt fokuserar på det jag ser som steg 2 i valideringen: att kontrollera om datans innehåll faktiskt är rimligt och följer uppsatta affärsregler. För att testa detta i praktiken har jag byggt en pipeline som granskar simulerad avbrottsstatistik från elnätet.

För att hålla projektet välvärdat har jag medvetet valt bort:

- Att bygga användargränssnitt eller dashboards.
- Att träna maskininlärningsmodeller.
- Realtidsvalidering med Pydantic (fokus ligger helt på batchdata i Pandas och Pandera).
- Externa databaser och molnlösningar (allt körs lokalt med strukturerad data).

Området, dess relevans och koppling till yrkesrollen

Inom Data Science är bra datakvalitet en ren överlevnadsfråga: *garbage in, garbage out* (IBM, u.å.-b). Om grunddatan inte stämmer spelar det ingen roll hur avancerade modeller eller analyser vi bygger – resultatet blir ändå felaktigt.

Ett stort problem i praktiken är det som brukar kallas "tysta fel". En datatabell kan se helt perfekt ut rent teknisk: alla kolumner har rätt datatyp, formaten stämmer och det finns inga saknade värden. Ändå kan innehållet vara helt orimligt (Bantilan, 2020; IBM, u.å.-a; IBM, u.å.-b). I mitt case med avbrottsdata kan det exempelvis handla om att ett avbrott har en sluttid som inträffar före starttiden, eller att ett planerat underhållsarbete har fått orsakskoden "storm". För ett elnätsbolag ställer sådana fel till det direkt i beräkningen av nyckeltal (som SAIDI och SAIFI) och vid utbetalning av lagstadgad avbrottsersättning.

Tidigare har vi ofta kontrollerat data genom att skriva egna if/else-funktioner för hand, loopa igenom kolumner och lägga felmeddelanden i en lista. Det fungerar bra för små skript, men i takt med att reglerna växer blir koden snabbt rörig, lång och svår att underhålla.

Pandera löser detta genom att låta oss sätta upp deklarativa datakontrakt (*Data Contracts*) direkt mot våra DataFrames (Reliable Data Engineering, 2026). Istället för att beskriva *hur* datorn ska leta fel steg för steg beskriver vi bara *vad* som ska gälla. Genom att sätta upp detta som en automatiserad kvalitetsgrind (*Quality Gate*) tidigt i dataflödet slipper vi som data scientists lägga onödig tid på att manuellt leta efter trasig data längre fram i analysarbetet (Bantilan, 2025; Odoo, 2026).

Viktiga begrepp

För att förstå hur projektet fungerar är det några centrala begrepp som är bra att reda ut:

- **Validering kontra städning:** Datavalidering är en kontrollstation som granskar datan mot uppsatta regler utan att ändra på den (Odoo, 2026). Datastädning är när man aktivt ändrar, korrigerar eller fyller i saknade värden (Odoo, 2026). Pipelinen i det här projektet är en ren kvalitetsgrind: den avgör bara vad som är rätt och fel, men manipulerar inga observationer.
- **Deklarativ vs. imperativ logik:** Imperativ kod handlar om att skriva manuella loopar och villkor för att kontrollera värden. Deklarativ logik innebär att vi definierar ett färdigt schema med regler och låter biblioteket sköta själva genomgången.
- **DataFrameSchema, Column och Check:** Det här är Panderas grundstengar. DataFrameSchema håller ihop hela schemat. Column anger strukturen och grundläggande krav (som datatyp och unika värden) medan Check utför de faktiska testerna på innehållet (som tillåtna intervall eller kategorier) (Statology, 2026; Towards Data Science, 2024).
- **Flerkolumnsvalidering (Wide Checks):** Vanliga kontroller granskar bara en kolumn i taget. Flerkolumnskontroller placeras direkt under schemat (checks=[]) och tar in hela tabellen som argument, vilket gör det möjligt att jämföra värden mellan olika kolumner (Bantilan, 2020).
- **Fail-Fast vs. Lazy Validation:** Standardläget i Pandera är att kasta ett SchemaError och krascha direkt vid första felaktiga cellen. Med parametern lazy=True går Pandera istället igenom hela datasetet och samlar alla avvikelser i ett SchemaErrors-objekt. Ur detta objekt kan vi sedan plocka ut tabellen failure_cases med alla fel och deras radindex.
- **Karantänhantering (Triage):** Istället för att kasta hela filen vid ett fel eller tyst filtrera bort rader, sorteras datan upp i tre delar: rena rader, avvisade rader (karantän) och en fellogg som förklarar varför raderna avvisades.

Genomförande och resultat

Jag valde att hålla projektstrukturen enkel och funktionsbaserad framför klassbaserad för att den ska vara lättläst och fungera bra som en ren demo av Pandera: Projektet är uppdelat i:

1. src/schemas.py: där schemat och alla affärsregler bor.
2. src/pipeline.py: en funktion som kör hela flödet.
3. main.py: startpunkten som ställer in loggningen och kör igång pipelinen.
4. tests/test_pipeline_errors.py: automatiserade tester med pytest

Konstanter och Single Source of Truth

För att undvika "magic numbers" och "magic strings" i koden lyfte jag ut alla branschregler och gränsvärden som konstanter högst upp i schemas.py:

```

13 # Domänkonstanter och tillåtna värdemängder
14 ALLOWED_VOLTAGE_LEVELS: Final[list[float]] = [0.4, 10.0, 20.0]
15 ALLOWED_OUTAGE_TYPES: Final[list[str]] = ["planerat", "oplanerat"]
16 ALLOWED_CAUSE_CATEGORIES: Final[list[str]] = [
17     "tekniskt_fel",
18     "underhåll",
19     "väder",
20     "grävskada",
21     "personal",
22     "okänd"
23 ]
24 MAX_VALID_TIMESTAMP: Final[pd.Timestamp] = pd.Timestamp("2026-06-30 23:59:59")
25 COMPENSATION_THRESHOLD_MINUTES: Final[int] = 720

```

Jag typannoterade dessa med Final från Pythons typing-modul (PEP 591). Python stoppar inte ändringar av variabler i själva körningen, men Final gör reglerna låsta för linters och statiska analysverktyg som Mypy vilket minskar risken att någon råkar skriva över en viktig regel längre ner i koden.

Logik och flerkolumnsregler (PEP 8)

Istället för att skriva anonyma lambdor bröt jag ut alla flerkolumnskontroller till namngivna funktioner med docstrings och typannoteringar. Det följer riktlinjerna i PEP 8 och gör att eventuella felrapporter för tydliga funktionsnamn samtidigt som reglerna går att enhetstesta separat.

Ett bra exempel är relationen mellan avbrottstyp och orsak: ett planerat avbrott måste bero på underhåll och ett underhållsarbete ska alltid vara planerat. Istället för krångliga villkor löste jag det med logisk ekvivalens:

```

33 def check_outage_cause_relation(df: pd.DataFrame) -> pd.Series:
34     """
35     Validerar sambandet mellan avbrottstyp och orsakskod:
36     - Ett planerat avbrott måste ha orsakskoden 'underhåll'.
37     - Ett oplanerat avbrott får inte ha orsakskoden 'underhåll'.
38     """
39     is_planned = df["outage_type"] == "planerat"
40     is_maintenance = df["cause_category"] == "underhåll"
41     return is_planned == is_maintenance

```

Eftersom detta körs vektoriserat i Pandas/NumPy jämförs serierna rad för rad direkt i C-kod:

- Om båda är True (planerat och underhåll) blir raden True (godkänd).
- Om båda är False (oplanerat och t.ex. väder) blir raden också True (godkänd).
- Om värdena inte matchar blir resultatet False, och Pandera underkänner raden.

På samma sätt kontrolleras lagkravet för avbrottsersättning i `check_compensation_eligibility`: flaggan ska bara vara True om avbrottet var oplanerat och pågick i minst 720 minuter (12 timmar):

```
43 def check_compensation_eligibility(df: pd.DataFrame) -> pd.Series:
44     """
45     Validerar affärslogiken för avbrottsersättning:
46     Ersättning är endast tillåten om avbrottet är oplanerat OCH varar >= 720 minuter.
47     """
48     qualifies_for_compensation = (df["outage_type"] == "oplanerat") & (
49         df["duration_minutes"] >= COMPENSATION_THRESHOLD_MINUTES
50     )
51     return df["compensation_eligible"] == qualifies_for_compensation
```

Själva schemat (outage_schema)

Schemat knyter ihop alla delar och är strikt definierat (`strict=True`) så inga oväntade extra kolumner kan slinka med. Några viktiga delar i schemat:

- `incident_id`: har `unique=True` för att stoppa dubblettregistreringar.
- `voltage_level_kv`: tillåter bara de fasta lokalnätensnivåerna 0,4, 10,0 och 20,0 kV.
- `start_time` och `end_time`: använder `coerce=True` för att parsas datumsträngar och spärras mot framtida datum med `MAX_VALID_TIMESTAMP`.
- `customers_affected` och `duration_minutes`: spärras mot orimliga negativa värden (`ge(0)`).

Hur pipelinen körs och sorterar felen

Huvudfunktionen `run_validation_pipeline()` i `src/pipeline.py` är orkestreraren som binder ihop hela processen i fyra steg:

1. Filkontroll: Kontrollerar att filen finns med `pathlib.Path.is_file()`. Saknas den loggas den absoluta sökvägen via `.resolve()` och skriptet reser ett `FileNotFoundError` direkt.
2. Säker inläsning: Fångar upp två olika kantfall för tomma filer: en 0-byte-fil kastar `EmptyDataError`, medan en fil med enbart rubriker fångas av `raw_df.empty` där pipelinen loggar en varning och avbryter utan krasch.
3. Validering och felhantering: Kör `outage_schema.validate(raw_df, lazy=True)` i ett try-block. Om fel upptäcks kliver koden in under `except pa.errors.SchemaErrors as exc`. Här gör koden en viktig uppdelning:
 - Strukturella fel: Koden kollar `errors_df["index"].isna().any()`. Om en hel obligatorisk kolumn saknas i CSV-filen kan inte Pandera sätta ett radnummer (index blir NaN). Då avvisas hela datasetet direkt.
 - Innehållsfel på radnivå: Om det bara är enstaka värden som är felaktiga plockas de unika felindexen ut. Med `raw_df.drop()` skapas `clean_df` (alla felfria rader) och med `raw_df.loc[ ]` skapas `rejected_df` (de avvisade raderna).
4. Spara filer: Målmappen skapas säkert med `makedirs(parents=True, exist_ok=True)` och filerna skrivs till disk med skydd under ett `except OSError`-block.

Resultatet blir tre tydliga filer i utdatamappen:

- `clean_outages.csv`: All data som klarade valideringen och kan användas till analys.
- `rejected_outages.csv`: Rader som avvisats och placerats i karantän.
- `failure_cases.csv`: Ett detaljerad relrapport som visar exakt vilket radindex, vilken kolumn, vilken regel och vilket felaktigt värde som orsakade avvikelsen.

Ett utdrag ur felrapporten (`failure_cases.csv`):

```
1 index,column,check,failure_case
2 0,incident_id,field_uniqueness,INC-2026-001
3 1,duration_minutes,duration_minutes kan inte vara negativ,-120
4 3,voltage_level_kv,"isin([0.4, 10.0, 20.0])",130.0
5 5,customers_affected,Konflikt mellan outage_type och cause_category,8
6 5,incident_id,Konflikt mellan outage_type och cause_category,INC-2026-020
7 5,cause_category,Konflikt mellan outage_type och cause_category,väder
```

Verifiering med tester (pytest)

För att kontrollera att felhanteringen i pipeline faktiskt fungerade i praktiken byggde jag tre automatiserade enhetstester i `tests/test_pipeline_errors.py` med `pytest`s inbyggda `tmp_path`-fixture:

1. `test_missing_input_file_raises_not_found`: Testar att funktionen reagerar med ett `FileNotFoundError` om filen inte existerar.
2. `test_empty_input_file_raises_empty_data_error`: Skapar en tom 0-byte-fil och verifierar att `EmptyDataError` reses som förväntat.
3. `test_missing_required_column_raises_schema_error`: Testar att pipeline hanterar strukturella fel korrekt. När en kolumn (`incident_id`) togs bort ur filen kraschade inte pipeline utan fångade felet, avvisade raden till karantän och bekräftade att felkoden `column_in_dataframe` fanns i felrapporten.

Begränsningar och möjliga förbättringar

Även om Pandera fungerat väldigt smidigt för den här typen av dataflöden finns det några tydliga begränsningar värda att lyfta:

- **Minnesanvändning (In-memory)**: Pandera körs i minnet ovanpå Pandas. När man kör med `lazy=True` hålls alla avvikelser och rader i arbetsminnet. För riktigt stora datamängder kan detta bli en flaskhals. Där vore det bättre att använda Panderas stöd för mer minneseffektiva motorer som Polars eller PySpark (Bantilan, 2025).
- **Batch vs realtid**: Den här lösningen är byggd för tabulär data som körs i batcher. För strömmande realtidsdata (till exempel driftinfo som ramlar in rad för rad i ett API) är Pydantic ett mycket mer naturligt val.
- **Tolerat datumparsning (`coerce=True`)**: Att sätta `coerce=True` är bekvämt eftersom Pandera sköter konverteringen av datumsträngar, men det är också väldigt förlåtande. Det finns en risk att felformaterad datum tolkas felaktigt. En förbättring hade varit att

lägga på en strikt regex-filter som förs kontrollera att formatet är ren ISO-8601 innan konvertering görs.

- **Uppdelning av ansvarsområden (Separation och Concerns):** I nuläget har jag samlat filkontroll, inläsning, schemavalidering och sparning i en och samma funktion (`run_validation_pipeline`). I ett större produktionsprojekt hade det varit renare att dela upp dessa steg i separata, fristående funktioner.
- **CI/CD och automatisering:** Nästa naturliga steg hade varit att schemalägga valideringen i ett orkestreringsvektyg (som Airflow) eller köra valideringen som en automatisk kontroll i en driftpipeline (Bantilan, 2025).
- **Begränsat stöd för rad-mot-rad-kontroller:** Pandera är byggt för att granska kolumner radvis, inte för att jämföra olika rader med varandra. Ska man kontrollera sekvenser – sin att två tidsintervall inte överlappar – måste man skriva manuell Pandas-kod. Bantilan (2020) lyfter själv detta som en svaghet i biblioteket och i en triage-pipeline blir det dessutom svårt att avgöra vilken av två krockade rader som faktiskt ska skickas till karantän.

Självreflektion

Vad lärde du dig som du inte kunde innan?

Att man inte behöver lära sig ett helt nytt sätt att tänka för att börja med datakontrakt, utan att Pandera låter en använda vanlig, hederlig Pandas-kod fast på ett mycket smartare sätt. Jag lärde mig också hur smidigt det är att använda `Final` från Pythons `typing`-modul för att spärra viktiga branschregler mot slarv i koden eller vid framtida vidareutveckling.

Vad var svårast att förstå eller genomföra?

Själva Pandera-koden var inte det svåra, utan snarare att sätta gränserna för mitt eget projekt. I verkligheten sitter man ju inte själv och hittar på både hur rådatan ska se ut in och vilka krav som ska ställas ut; det är något som bestäms tillsammans med domänexperter och de som ska använda analysen. Här fick jag vara både beställare, elnätskunnig och programmerare samtidigt, vilket gjorde det svårt att veta exakt var man skulle dra gränsen för vad som är en rimlig standard.

Vilket tekniskt val är du mest nöjd med och varför?

Min karantänlösning med de tre filerna (`clean`, `rejected` och `failure_cases`). Det känns som en mycket mer användbar metod i verkligheten än att bara låta hela skriptet krascha så fort en enda rad är fel. Jag är också väldigt nöjd med flerkolumnskontrollerna.

Vad hade du gjort annorlunda om du började om?

Hade jag börjat om från början med det jag vet idag hade jag byggt upp strukturen lite mer som ett riktigt produktinsflöde direkt. Det vill säga inte samla filhantering, inläsning och validering i en och samma funktion i `pipeline.py`, utan dela upp det i fristående delar. Jag hade nog också lagt till en regex-kontroll på datumfälten innan Pandera får gissa format med `coerce=True`.

Vad skulle vara ett naturligt nästa steg om du fortsatte arbetet?

Att sätta upp skriptet så att det körs automatiskt i en schemalagd pipeline eller ett orkestreringsverktyg som Airflow. Sedan hade det varit väldigt roligt att testa hur Pandera beter sig mot mer minneseffektiva verktyg som Polars eller PySpark när datamängderna växer till miljontals rader.

Vilket betyg tycker du själv att arbetet motsvarar – G eller VG? Motivera

Jag tycker mitt arbete motsvarar kraven för VG. Utöver att uppfylla alla grundkrav för G tycker jag att jag visar en fördjupad förståelse genom följande punkter:

- **Hur och varför tekniken fungerar:** Jag nöjer mig inte med att bara visa att Pandera fungerar, utan förklarar vad som sker under huven – till exempel hur logisk ekvivalens exekveras vektoriserat i C-cod på Pandas-serier eller hur lazy=True ackumulerar fel i minnet.
- **Egna tekniska val och motiveringar:** Jag har tagit aktiva beslut genom hela bygget: att välja funktioner framför klasser för tydlighetens skull, att skriva namngivna flerkolumnskontroller med docstrings enligt PEP 8 istället för lambdor och att bygga ett triage-mönster som räddar godkända rader istället för att bara kasta hela filen.
- **Välskriven kod och hantering av kantfall:** Koden är inte bara ett enkelt demo, utan fångar verkliga problem som tomma 0-byte-filer, filer med enbart rubriker och strukturella fel där kolumner saknas. Alla dessa scenarier har jag dessutom verifierat med automatiserade enhetstester i pytest.
- **Pedagogiskt exempel:** Jag har använt ett verklighetsnära case med avbrottsdata från elnätet för att tydligt visa varför vanliga typkontroller inte räcker och hur domänspecifika regler och korsvalidering mellan kolumner stoppar tysta fel.
- **Självständigt resonemang om begränsningar och alternativ:** Jag har lyft lösningens faktiska svagheter, som minnesåtgången vid stora datamängder och riskerna med coerce=True. Dessutom har jag resonerat kring när tekniken passar (tabulär batchdata) och när andra verktyg är bättre, till exempel Pydantic för realtidsströmmar eller Polars/PySpark för storskalig data.

Källor

- **Bantilan, N. (2020).** pandera: Statistical Data Validation of Pandas Dataframes. I: Proceedings of the 19th Python in Science Conference (SciPy 2020).
<https://doi.org/10.25080/Majora-342d178e-010>
- **Bantilan, N. (2025).** Intro to Data Validation with Pandera - Data Quality Workshop. Union.ai. <https://www.youtube.com/watch?v=Yb6yTdTC7gk>
- **IBM. (u.å.-a).** Data Quality Dimensions. IBM Think Topics.
<https://www.ibm.com/think/topics/data-quality-dimensions>
- **IBM. (u.å.-b).** What is Data Validation? IBM Think Topics.
<https://www.ibm.com/think/topics/data-validation>
- **Odoo. (2026).** Data Validation and Data Cleaning Techniques. Online Tech Course Blog.
<https://online-tech-course.odoo.com/blog/course-1/data-validation-and-data-cleaning-techniques-16>
- **Pandera Documentation. (u.å.).** DataFrameSchemas and Checks.
<https://pandera.readthedocs.io/>
- **Python Software Foundation. (2001).** PEP 8 – Style Guide for Python Code.
<https://peps.python.org/pep-0008/>
- **Python Software Foundation. (2019).** PEP 591 – Adding a final qualifier to typing.
<https://peps.python.org/pep-0591/>
- **Reliable Data Engineering. (2026).** 20 Data Validation Rules Every Pipeline Should Have. Medium.
<https://medium.com/@reliabledataengineering/20-data-validation-rules-every-pipeline-should-have-2e332e4f74eb>
- **Statology. (2026).** Data Validation in Python with Pandera: A Practical Introduction.
<https://www.statology.org/data-validation-in-python-with-pandera-a-practical-introduction/>
- **Towards Data Science. (2024).** How to Easily Validate your Data with Pandera.
<https://towardsdatascience.com/how-to-easily-validate-your-data-with-pandera-a9cd22c515a5/>